Diseño e Implementación de una Plataforma Web Editorial Multicanal Basada en el Patrón Arquitectónico Modelo-Vista-Controlador (MVC)

[*]Proyecto de Cátedra – Hackathon de Arquitectura de Sistemas

Marcos Josué García Maldonado
Facultad de Ingeniería
Universidad Mesoamericana
Carné: 202408055
marcosjgm@umes.edu.gt

Arturo Alejandro Maldonado Orozco
Facultad de Ingeniería
Universidad Mesoamericana
Carné: 2023080110
20arturomaldonado@umes.edu.gt

Sebastián Rodrigo Rodas Pineda
Facultad de Ingeniería
Universidad Mesoamericana
Carné: 202408060
srodas477@umes.edu.gt

Resumen—El presente artículo expone el diseño, desarrollo e implementación técnica de una Plataforma Web Editorial Multicanal concebida bajo el marco de la Hackathon de Arquitectura de Sistemas. El sistema atiende la complejidad operativa de la industria editorial contemporánea mediante la unificación de canales de venta minorista y transacciones corporativas B2B. La arquitectura adoptada se cimenta en la aplicación rigurosa del patrón arquitectónico Modelo-Vista-Controlador (MVC), asegurando una separación estricta entre la capa de presentación, la orquestación de servicios y el dominio de negocio. Se integró un motor matemático de descuentos dinámicos ejecutado exclusivamente en la capa del Modelo, un esquema de seguridad basado en roles (RBAC) con cinco perfiles de acceso, un protocolo de idempotencia criptográfica dual (RAM y PostgreSQL) mediante firmas HMAC-SHA256, un limitador temporal de checkout de 10 minutos con sincronización reactiva vía WebSockets (Socket.IO), y almacenamiento en la nube de recursos multimedia con Cloudinary. La solución garantiza alta disponibilidad, mantenibilidad, modularidad y apego a las directrices de integridad transaccional ACID.

Index Terms—Modelo-Vista-Controlador (MVC), Arquitectura de Software, Idempotencia Criptográfica, WebSockets, Control de Acceso Basado en Roles (RBAC), PostgreSQL, Node.js.

I. Introducción

La transformación digital de la industria editorial demanda ecosistemas de software con alta capacidad de adaptación para gestionar inventarios bibliográficos distribuidos, modelos de precios heterogéneos y mecanismos de venta que abarcan tanto al consumidor individual como a clientes institucionales (universidades, colegios y redes de bibliotecas). La problemática central radica en que los sistemas de comercio electrónico convencionales suelen acoplar indebidamente la lógica de cálculo financiero con los controladores HTTP o las plantillas de vista, derivando en arquitecturas monolíticas frágiles, de difícil depuración y propensas a inconsistencias en auditorías contables.

Ante este desafío, la Hackathon de Arquitectura de Sistemas plantea el desarrollo de una plataforma web integral cuya premisa irremplazable es el cumplimiento estricto del patrón arquitectónico Modelo-Vista-Controlador (MVC). En este marco, el sistema no solo debe solventar la navegación de catálogo y el carrito de compras, sino gobernar un flujo de cotizaciones comerciales B2B, aplicar matrices dinámicas de descuento acumulativo, prevenir la duplicidad transaccional en ambientes de alta concurrencia mediante idempotencia garantizada, y limitar la persistencia temporal de sesiones de compra a un horizonte no mayor a 10 minutos con retroalimentación en tiempo real.

El presente artículo documenta la arquitectura global del sistema, desglosando la estructura de entidades de datos, la encapsulación matemática en los modelos, el esquema de autorización y los mecanismos de comunicación bidireccional implementados.

II. Arquitectura del Sistema (Patrón MVC)

II-A. Separación Estricta de Responsabilidades

Para salvaguardar los principios de alta cohesión y bajo acoplamiento, la plataforma se estructura en tres capas conceptualmente aisladas:

1. Capa Modelo (Model): Constituye el núcleo de la aplicación. Alberga las definiciones de entidad, el esquema relacional mapeado a través del ORM Sequelize, las restricciones de integridad y, primordialmente, las reglas dinámicas de negocio. Siguiendo las directrices del proyecto, queda terminantemente prohibida la inclusión de elementos de interfaz gráfica (HTML), manipulación directa del protocolo HTTP o delegación de cálculos contables a capas externas.
2. Capa Controlador (Controller): Actúa como director y orquestador. Captura las solicitudes entrantes (HTTP/REST y eventos WebSocket), delega la ejecución de operaciones analíticas o transaccionales

a los Modelos pertinentes, y emite respuestas en formatos estructurados estándar (JSON). No contiene sentencias SQL manuales ni lógica de liquidación financiera.

3. Capa Vista (View): Responsable exclusiva de la presentación y experiencia de usuario. Opera de forma completamente desacoplada de la base de datos, consumiendo únicamente los datos entregados por los controladores para renderizar interfaces responsivas.

II-B. Topología Arquitectónica

La Fig. 1 ilustra el flujo de comunicación y el desacoplamiento entre las capas de la plataforma:



Figura 1. Topología y Arquitectura Multicapa del Sistema Editorial.

III. Diseño y Modelo de Base de Datos

La persistencia de la información descansa sobre el motor relacional PostgreSQL (v14+). El esquema está compuesto por 18 tablas normalizadas en Tercera Forma Normal (3FN), garantizando integridad referencial mediante claves foráneas con políticas de actualización y eliminación en cascada controladas.

III-A. Entidades Fundamentales

- Gestión Bibliográfica: Estructurada mediante las tablas libros, autores, editoriales, generos y formatos. Se implementan tablas asociativas N:M (libros_autores, libros_editoriales, libros_generos) para representar la multiplicidad de participantes en una obra.
- Catálogo y Variantes: La tabla catalogo_productos vincula el libro con su formato físico o digital (Tapa Blanda, Tapa Dura, E-Book, Audiolibro), gestionando el ISBN, precio base y el stock disponible para despacho.
- Seguridad y Usuarios: Conectada mediante usuarios, roles y perfiles_institucionales. Esta última almacena datos tributarios y de acreditación corporativa (NIT, razón social y sector de actividad).
- Ventas y Transacciones: Conformada por carritos, carrito_items, ordenes, orden_detalles, checkout_sesiones e idempotencias.
- Cotizaciones Comerciales B2B: Modelada a través de cotizaciones_solicitudes y cotizaciones_propuestas, soportando el ciclo de negociación entre el cliente institucional y el asesor de ventas.

IV. Lógica de Negocio y Motor de Descuentos

En estricto apego al requerimiento funcional RF-03.2, la lógica de cálculo financiero fue encapsulada en el método estático Carritos.calcularTotales del Modelo, eliminando cualquier procesamiento contable en los controladores.

IV-A. Matriz de Reglas Dinámicas

La formulación matemática de la liquidación opera según las siguientes ecuaciones:

$$S_{\text{base}} = \sum_{i=1}^{n} (P_i \times Q_i) \tag{1}$$

donde P_i representa el precio base unitario y Q_i la cantidad demandada del ítem i. El descuento por volumen de compra (D_{vol}) se asigna escalonadamente en función del volumen total $U = \sum Q_i$:

$$\%_{\text{vol}} = \begin{cases} 0\,\%, & \text{si } 1 \leq U \leq 5 \\ 5\,\%, & \text{si } 6 \leq U \leq 20 \\ 10\,\%, & \text{si } 21 \leq U \leq 50 \\ 15\,\%, & \text{si } U > 50 \end{cases} \tag{2}$$

$$D_{\text{vol}} = S_{\text{base}} \times \%_{\text{vol}} \tag{3}$$

El subtotal resultante tras la aplicación del volumen (S_{vol}) sirve de base para el descuento institucional acumulativo (D_{inst}), exclusivo para entidades educativas o comerciales acreditadas:

$$S_{\text{vol}} = S_{\text{base}} - D_{\text{vol}} \tag{4}$$

$$D_{\text{inst}} = \begin{cases} S_{\text{vol}} \times 0.08, & \text{si Rol} = \text{Institucional} \\ 0, & \text{si Rol} \neq \text{Institucional} \end{cases} \tag{5}$$

Finalmente, el descuento global (D_{total}), la base imponible neta (S_{neto}), el Impuesto al Valor Agregado (IVA, $12\,\%$) y el valor de liquidación definitivo (T_{final}) se obtienen según:

$$D_{\text{total}} = D_{\text{vol}} + D_{\text{inst}} \tag{6}$$

$$S_{\text{neto}} = S_{\text{base}} - D_{\text{total}} \tag{7}$$

$$IVA = S_{\text{neto}} \times 0.12 \tag{8}$$

$$T_{\text{final}} = S_{\text{neto}} + IVA \tag{9}$$

La Tabla I resume los escenarios de prueba ejecutados para corroborar la precisión de la lógica en el Modelo.

Cuadro I
Matriz de Liquidación y Descuentos Dinámicos

Escenario	Unidades	% Vol.	% Inst.	Total Final (Q)
Minorista Individual	2	0 %	0 %	Base + IVA
Minorista Mediano	10	5 %	0 %	Base - 5 % + IVA
Institucional Básico	4	0 %	8 %	Base - 8 % + IVA
Institucional Medio	15	5 %	8 % acum.	Descuento Mixto
Institucional Masivo	60	15 %	8 % acum.	Beneficio Máximo

V. Seguridad y Control de Acceso (RBAC)

El sistema implementa un modelo de Control de Acceso Basado en Roles (RBAC) granular que clasifica a los intervinientes en cinco categorías claramente tipificadas:

1. Visitante (Anónimo): Consulta abierta de catálogo, landing y fichas técnicas. No puede operar carritos ni iniciar compras.
2. Cliente Minorista: Compras a nivel individual, gestión de carrito propio e historial de órdenes personales.
3. Cliente Institucional: Acceso a la matriz de precios corporativos (+8 % acumulativo) y emisión de solicitudes de pedidos especiales no disponibles en catálogo común.
4. Asesor de Ventas: Acceso a la bandeja de cotizaciones, capacidad de asignar expedientes, formulación de propuestas económicas y seguimiento comercial.
5. Administrador: Gobernanza completa del sistema, mantenimiento de libros y autores, ajuste de inventario, auditoría global y validación institucional.

Las credenciales de acceso se protegen mediante la función de dispersión unidireccional Bcrypt con un factor de trabajo (salt rounds) de 10. La autorización de endpoints se rige por tokens JSON Web Token (JWT) con separación de Access Token (vida útil corta) y Refresh Token.

VI. Idempotencia y Checkout en Tiempo Real

VI-A. Idempotencia Criptográfica Dual

Para prevenir problemas de doble cobro o deducción duplicada de inventario ante pérdidas de paquetes o reintentos del usuario, el sistema incorpora un esquema de idempotencia soportado en la variable de entorno IDEMPOTENCY_KEY. La llave de idempotencia se construye mediante HMAC-SHA256:

$$K_{\text{idemp}} = \text{HMAC-SHA256}(\text{Key}_{\text{env}}, \text{Payload}_{\text{tx}}) \qquad (10)$$

El mecanismo opera en dos capas concurrentes:

- Capa en Memoria RAM: Resuelve colisiones en milisegundos para peticiones concurrentes simultáneas.
- Persistencia en BD (idempotencias): Almacena la llave (VARCHAR(255)) junto con el payload de respuesta en formato JSONB. Si el servicio se reinicia, la solicitud retransmitida es reconocida y despachada con la cabecera X-Idempotency-Replay: true sin reinvocar la transacción en la base de datos.
- Restricción Unívoca: El campo ordenes.idempotency_key posee un constraint UNIQUE que blinda la consistencia a nivel de motor SQL.

VI-B. Sesiones de Checkout de 10 Minutos y WebSockets

El proceso de checkout establece una reserva temporal de inventario sujeta a una vigencia estricta de 600 segundos (10 minutos):

1. Al iniciar el checkout (POST /api/ordenes/iniciar-checkout), se persiste un registro en la tabla checkout_sesiones con los ítems y desgloses en JSONB, fijando la marca temporal expira_en.
2. El servidor activa un temporizador Socket.IO que transmite cada segundo el evento checkout:tick a la sala del usuario (checkout:{userId}).
3. En caso de sobrepasar los 10 minutos sin confirmar la orden, el estado muta a expirada, se emite checkout:expirado y la API responde con código 410 Gone, redireccionando forzosamente a la tienda.
4. Si el usuario confirma en tiempo, la orden se crea dentro de una transacción atómica gestionada por Sequelize (transaction.commit()), la sesión pasa a completada y se emite reporte:nueva_venta a los paneles de administración en tiempo real.

VII. Integraciones Externas y Documentación

VII-A. Almacenamiento Multimedia con Cloudinary

Para el almacenamiento de portadas de libros requeridas en el catálogo (models/Libros.js), se implementó una integración en la nube con Cloudinary respaldada por un middleware de procesamiento en memoria con Multer. La API permite tanto la subida directa de archivos binarios (multipart/form-data) como la asignación de URLs preexistentes, alojando las imágenes de forma optimizada mediante CDN global.

VII-B. Documentación Interactiva Swagger / OpenAPI 3.0

Todos los endpoints desarrollados se documentaron rigurosamente bajo el estándar OpenAPI 3.0 mediante Swagger UI. La interfaz interactiva se encuentra disponible en las rutas /api-docs y /docs, permitiendo la prueba autenticada de rutas mediante inyección de Bearer Tokens y encabezados de idempotencia.

VIII. Metodología de Desarrollo y Pruebas

El desarrollo del proyecto se condujo bajo la metodología ágil Scrum adaptada a hackathon, apoyada en un estricto control de versiones con Git mediante ramas funcionales (feature branches) unificadas sobre la rama principal Dev.

Se elaboraron baterías de pruebas de integración para validar:

- Consistencia de cálculo en el motor de descuentos ante carritos mixtos de 1 a 100 ejemplares.

- Resistencia a ataques de inyección SQL mediante sentencias parametrizadas del ORM.
- Verificación de caducidad temporal a los 600 segundos exactos en sesiones de checkout.
- Comportamiento idempotente ante ráfagas de 5 peticiones simultáneas con idéntico identificador.

Los resultados confirmaron tiempos de respuesta promedio inferiores a 120 ms en consultas locales y total integridad referencial en las 18 tablas relacionales.

IX. Conclusiones

La aplicación sistemática del patrón arquitectónico Modelo-Vista-Controlador (MVC) posibilitó la creación de una plataforma editorial sólida, escalable y mantenible. Se demostró que la delegación exclusiva de las reglas financieras a la capa del Modelo protege la lógica de dominio ante cambios en la infraestructura de transporte o presentación. La combinación de transacciones ACID en PostgreSQL, doble persistencia de idempotencia (HMAC-SHA256) y sincronización reactiva bidireccional mediante WebSockets garantiza un flujo de ventas robusto, capaz de soportar operaciones masivas sin riesgo de sobreventa de inventario ni transacciones duplicadas.

Referencias

[1] E. Gamma, R. Helm, R. Johnson, y J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA, USA: Addison-Wesley, 1994.

[2] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

[3] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston, MA, USA: Prentice Hall, 2017.

[4] IEEE Standard for Software Quality Assurance Processes, IEEE Std 730-2014, 2014.

[5] M. Jones, J. Bradley, y N. Sakimura, "JSON Web Token (JWT)", RFC 7519, Internet Engineering Task Force (IETF), Mayo 2015.

[6] I. Fette y A. Melnikov, "The WebSocket Protocol", RFC 6455, Internet Engineering Task Force (IETF), Diciembre 2011.

[7] J. Postel, "Transmission Control Protocol", RFC 793, Internet Engineering Task Force (IETF), Septiembre 1981.